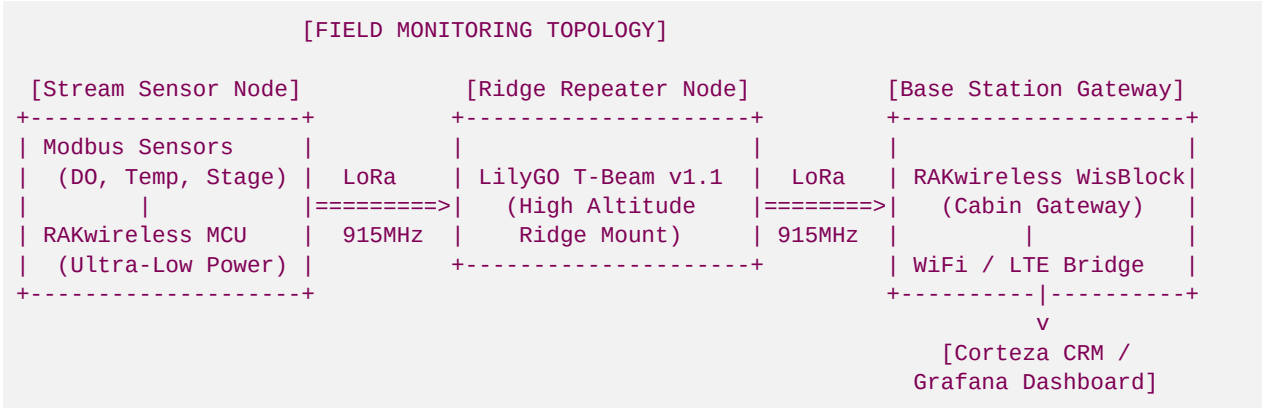
File 56: Off-Grid LoRa Meshtastic Stream Monitoring & Alibaba Hardware Integration Guide

> ***Off-Grid Telemetry and Hydrological Monitoring Standard**: This document establishes the technical blueprint and sourcing guide for integrating open-source **Meshtastic (LoRa 915 MHz)** mesh networking with industrial **RS485 Modbus RTU** environmental sensors sourced directly from Alibaba manufacturers. High-gradient mountain streams in North Georgia (such as the Upper Toccoa, Cartecay, and Etowah basins) are frequently located in deep gorges with zero cellular or satellite coverage. To satisfy the USACE Savannah District's strict 7-year monitoring and credit-release guidelines (requiring continuous water temperature, dissolved oxygen, turbidity, and stage-level logs), this framework provides Hunter Morris and his field crew with a self-hosted, solar-powered, off-grid telemetry network costing less than $100 per sensor node.*

I. System Architecture: Off-Grid Stream Telemetry

Fluvial bioengineering and geomorphic stream restoration projects require continuous water quality and hydrological validation to ensure wild trout spawning habitat remains active (water temperatures must stay strictly below 65°F or 18.3°C with dissolved oxygen >= 6.0 mg/L).
Instead of expensive commercial telemetry systems that lock data behind proprietary cloud services, we leverage **Meshtastic**, a decentralized, off-grid, peer-to-peer LoRa mesh communications framework operating on the unlicensed 915 MHz ISM band in North America:



Key Architectural Layers:

1. **Stream Sensor Node**: Placed streamside in water-resistant NEMA-4X enclosures. It periodically wakes up from deep sleep, powers the environmental sensors, polls their data registers via RS485, packs the values, transmits them as a LoRa packet, and returns to sleep.
2. **Ridge Repeater Node**: Solar-powered LilyGO T-Beam or Heltec V3 router nodes mounted high up on ridges to overcome topography blockages (such as mountain stream gorges) and extend line-of-sight range up to 10+ miles.

3. **Base Station Gateway**: Located at Hunter's cabin or a local base station with internet connectivity. It receives telemetry packets via LoRa, converts them to MQTT messages over local WiFi, and uploads them to our self-hosted **Corteza CRM** and a **Grafana** dashboard.

II. Sourcing: Alibaba Industrial Modbus Sensors & Transceivers

For stream immersion, consumer-grade analog sensors (like typical Arduino hobbyist probes) drift rapidly, corrode, and suffer from massive signal loss over long cables. We utilize **RS485 Modbus RTU** industrial sensors. Modbus RTU transmits digital signals using differential voltages over twisted-pair wires, allowing cables to run up to 1,200 meters through rugged undergrowth with zero signal degradation.
We source these high-durability, low-cost sensors directly from top-tier Alibaba industrial manufacturers:

1. Submersible Hydrostatic Water Level (Stage) Transducer

- **Supplier**: *Dalian Taijia Technology Co., Ltd.* (Alibaba Gold Supplier)
- **Search Query**: `RS485 Modbus Submersible Water Level Transmitter`
- **Key Specs**:
- 316L Stainless Steel housing with a polyurethane vented cable (containing a hollow capillary tube to compensate for atmospheric pressure changes).
- Output: RS485 Modbus RTU (Registers: `0x0004` for pressure, `0x0005` for temperature).
- Accuracy: ±0.25% Span.
- Range: 0–3 meters (ideal for mountain stream pools).
- **Estimated Cost**: $38.00–$45.00 USD (vs. $450.00 commercial equivalent).

2. Optical Dissolved Oxygen (DO) Electrode

- **Supplier**: *Zhengzhou Pinsheng Electronic Technology Co., Ltd. (Desun Uniwill)*
- **Search Query**: `RS485 Modbus Optical Dissolved Oxygen Sensor`
- **Key Specs**:
- Luminescent lifetime technology (does not consume oxygen, requires zero electrolyte replenishment, and is unaffected by flow velocity).
- Built-in automatic temperature compensation (NTC thermistor).
- Output: Modbus RTU (Registers: `0x0001` for DO concentration in mg/L, `0x0002` for temperature).
- Range: 0–20 mg/L (Accuracy: ±0.1 mg/L).
- **Estimated Cost**: $120.00–$145.00 USD (vs. $1,800.00 commercial equivalent).

3. Modbus Optical Turbidity Sensor

- **Supplier**: *Hunan Rika Electronic Tech Co., Ltd. (Rika Sensor)*
- **Search Query**: `Rika Turbidity Sensor RS485 Modbus`
- **Key Specs**:
- 90-degree scattered infrared light (ISO 7027 standard) to detect suspended soil sediment load and stream erosion.
- Includes a mechanical self-cleaning wiper brush to prevent biological fouling in wild mountain trout waters.

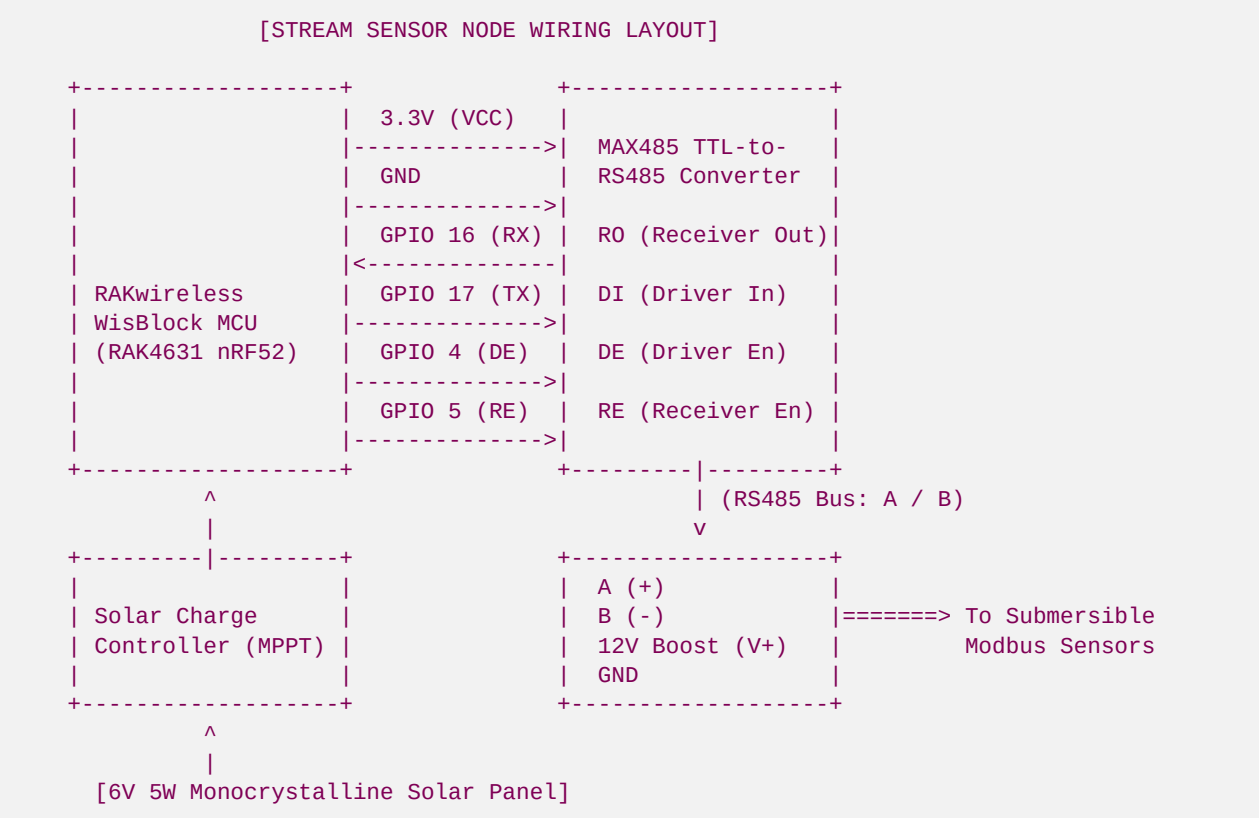
- Output: Modbus RTU (Register: `0x0000` for turbidity in NTU).
- Range: 0–1000 NTU (Accuracy: ±5%).
- **Estimated Cost**: $160.00–$190.00 USD.

4. Ultra-Low Power LoRa Node & Transceiver (Alibaba Supplier)

- **Supplier**: *Shenzhen Ebyte Electronic Technology Co., Ltd.*
- **Search Query**: `Ebyte E22-900T22S 915MHz LoRa Module` or `Ebyte RS485 to LoRa Wireless Modem`
- **Key Specs**:
- Semtech SX1262 LoRa chip. High power output (22dBm / 160mW) with ultra-high receiver sensitivity.
- Low power consumption: < 2.0 µA in deep sleep.
- **Estimated Cost**: $4.50–$6.00 USD per raw module.

III. Hardware Schematic: Streamside Telemetry Node

To construct a self-contained, low-power telemetry node, we interface our RS485 Modbus sensors with an **ESP32** or **nRF52840** microcontroller using a **MAX485 TTL-to-RS485 Transceiver** chip. Since industrial sensors operate on 9V to 24V DC, while our microcontroller operates on 3.3V, we utilize a **3.7V** LiFePO4 battery combined with an efficient boost converter to provide power only when taking measurements.



Modbus Transceiver Control Sequence (DE/RE Pin Logic):

The MAX485 transceiver is a half-duplex chip. We must control when the microcontroller is transmitting (sending commands to sensors) versus receiving (reading data back):

- **To Send Command (TX)**: Pull **DE (Driver Enable)** and **RE (Receiver Enable)** HIGH. The chip is now in transmit mode. Send the Modbus request byte stream.
- **To Read Response (RX)**: Immediately pull **DE** and **RE** LOW. The chip is now in receive mode. Read the response bytes.

IV. Telemetry Data Compression & Transmission Code

LoRa network packets (especially on peer-to-peer networks like Meshtastic) must be kept as small as possible to minimize airtime transmission window limits and battery drain. Sending long JSON or text strings is highly inefficient.
Instead, we compress our geomorphic stream data (water temperature, dissolved oxygen, turbidity, and stage) into a **dense 8-byte binary hexadecimal packet**:

Data Packet Bit Allocation (8 Bytes / 64 Bits total):

Bytes	Metric	Range	Scale Factor	Resolution
0 - 1 (16 bits)	**Water Temperature**	-20.0°C to +50.0°C	x 100	0.01°C
2 - 3 (16 bits)	**Dissolved Oxygen**	0.00 mg/L to 20.00 mg/L	x 100	0.01 mg/L
4 - 5 (16 bits)	**Stream Stage (Level)**	0.000 m to 5.000 m	x 1000	1.0 mm
6 - 7 (16 bits)	**Water Turbidity**	0.0 NTU to 1000.0 NTU	x 10	0.1 NTU

Microcontroller Sourcing & Compression Script (C++ / Arduino IDE):

Compile and upload this low-power script to your stream monitoring nodes to read the sensors and transmit via Meshtastic Serial API:

```
#include <Arduino.h>
#include <SoftwareSerial.h>

// MAX485 Transceiver Pins
#define DE_PIN 4
#define RE_PIN 5

// Submersible Sensor Modbus Request Payloads (Standard Hex Byte Arrays)
const byte ModbusDORequest[]   = {0x01, 0x03, 0x00, 0x01, 0x00, 0x01, 0xD5, 0xCA}; // Addr
ess 1, Read DO Reg 0x0001
const byte ModbusTempRequest[] = {0x02, 0x03, 0x00, 0x04, 0x00, 0x01, 0xC5, 0xF8}; // Addr
ess 2, Read Temp Reg 0x0004
const byte ModbusLevelRequest[]= {0x03, 0x03, 0x00, 0x04, 0x00, 0x01, 0xC5, 0xC9}; // Addr
ess 3, Read Level Reg 0x0004
const byte ModbusTurbRequest[] = {0x04, 0x03, 0x00, 0x00, 0x00, 0x01, 0x84, 0x5F}; // Addr
```

```
ess 4, Read Turb Reg 0x0000

SoftwareSerial ModbusSerial(16, 17); // RX, TX pins

// Reads a single Modbus RTU register value
float readModbusRegister(const byte* request, int length) {
    digitalWrite(DE_PIN, HIGH);
    digitalWrite(RE_PIN, HIGH);
    delay(5);

    ModbusSerial.write(request, length);
    ModbusSerial.flush();

    digitalWrite(DE_PIN, LOW);
    digitalWrite(RE_PIN, LOW);

    byte response[8];
    memset(response, 0, sizeof(response));

    delay(100); // Wait for sensor response

    int index = 0;
    while (ModbusSerial.available() && index < 8) {
        response[index++] = ModbusSerial.read();
    }

    if (index >= 7 && response[1] == 0x03) { // Successful Read
        // Extract 16-bit register value from bytes 3 & 4
        int rawVal = (response[3] << 8) | response[4];
        return (float)rawVal;
    }
    return -999.0; // Return error state
}

void setup() {
    Serial.begin(115200);       // Debug USB Output
    ModbusSerial.begin(9600);    // Modbus RTU Standard Baud

    pinMode(DE_PIN, OUTPUT);
    pinMode(RE_PIN, OUTPUT);

    // Set transceiver to receive mode by default
    digitalWrite(DE_PIN, LOW);
    digitalWrite(RE_PIN, LOW);
}

void loop() {
    // 1. Poll the Modbus RTU sensors
    float rawTemp = readModbusRegister(ModbusTempRequest, 8) / 100.0;
    float rawDO   = readModbusRegister(ModbusDORequest, 8) / 100.0;
    float rawLevel = readModbusRegister(ModbusLevelRequest, 8) / 1000.0;
    float rawTurb  = readModbusRegister(ModbusTurbRequest, 8) / 10.0;

    // 2. Compress floating-point data to 16-bit unsigned integers
    uint16_t compTemp  = (uint16_t)((rawTemp + 20.0) * 100.0); // Offset to handle negativ
e temps
    uint16_t compDO    = (uint16_t)(rawDO * 100.0);
```

```
    uint16_t compLevel = (uint16_t)(rawLevel * 1000.0);
    uint16_t compTurb  = (uint16_t)(rawTurb * 10.0);

    // 3. Assemble binary telemetry payload
    byte packet[8];
    packet[0] = (compTemp >> 8) & 0xFF;
    packet[1] = compTemp & 0xFF;
    packet[2] = (compDO >> 8) & 0xFF;
    packet[3] = compDO & 0xFF;
    packet[4] = (compLevel >> 8) & 0xFF;
    packet[5] = compLevel & 0xFF;
    packet[6] = (compTurb >> 8) & 0xFF;
    packet[7] = compTurb & 0xFF;

    // 4. Output payload as hexadecimal string to Meshtastic Node Serial API
    Serial.print("TX_HEX:");
    for (int k = 0; k < 8; k++) {
        if (packet[k] < 0x10) Serial.print("0");
        Serial.print(packet[k], HEX);
    }
    Serial.println(); // Broadcast command to Meshtastic

    // 5. Deep Sleep for 15 minutes (900,000ms) to conserve solar cell reserves
    delay(900000);
}
```

V. Base Station Gateway: Decompression & Database Sync

When the 8-byte hex payload reaches the Base Station Gateway at Hunter's cabin, a local gateway script (such as a **Node-RED** or a **Python** worker process) catches the Meshtastic serial output, decompresses the hexadecimal values back into standard geomorphic metrics, and stores them in your SQL/NoSQL database:

Gateway Parser & Database Syncer (Python):

Deploy this decompression parser on the gateway server to automatically parse and map incoming hydrological streams:

```
import pymysql
import datetime

# Database Sourcing Settings (Matches Odoo/Corteza CRM from File 39)
DB_HOST = "localhost"
DB_USER = "corteza_user"
DB_PASS = "corteza_secure_pass"
DB_NAME = "corteza_crm"

def decode_meshtastic_telemetry(hex_str):
    """Decompresses the 8-byte hex packet back to real hydrological values."""
    if len(hex_str) != 16:
        print("Error: Invalid packet length.")
        return None

    # Convert hex string back to byte arrays
    packet = bytes.fromhex(hex_str)
```

```
    # 1. Decode Water Temperature (Bytes 0 & 1)
    comp_temp = (packet[0] << 8) | packet[1]
    water_temp = (comp_temp / 100.0) - 20.0

    # 2. Decode Dissolved Oxygen (Bytes 2 & 3)
    comp_do = (packet[2] << 8) | packet[3]
    dissolved_oxygen = comp_do / 100.0

    # 3. Decode Water Level (Bytes 4 & 5)
    comp_level = (packet[4] << 8) | packet[5]
    water_level = comp_level / 1000.0

    # 4. Decode Turbidity (Bytes 6 & 7)
    comp_turb = (packet[6] << 8) | packet[7]
    turbidity = comp_turb / 10.0

    return {
        "timestamp": datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
        "water_temp_c": round(water_temp, 2),
        "water_temp_f": round((water_temp * 9.0/5.0) + 32.0, 2),
        "dissolved_oxygen_mg_l": round(dissolved_oxygen, 2),
        "water_level_m": round(water_level, 3),
        "turbidity_ntu": round(turbidity, 1)
    }

def sync_to_corteza_crm(data):
    """Writes decompressed telemetry straight to the CRM project database."""
    try:
        connection = pymysql.connect(
            host=DB_HOST,
            user=DB_USER,
            password=DB_PASS,
            database=DB_NAME
        )
        with connection.cursor() as cursor:
            # SQL to insert stream monitoring data
            sql = """
            INSERT INTO stream_telemetry_logs
            (project_id, logged_at, temperature_f, dissolved_oxygen, stage_level_m, turbid
ity_ntu)
            VALUES (%s, %s, %s, %s, %s, %s)
            """
            # Anderson Creek flagship HUC project ID = 101
            cursor.execute(sql, (
                101,
                data["timestamp"],
                data["water_temp_f"],
                data["dissolved_oxygen_mg_l"],
                data["water_level_m"],
                data["turbidity_ntu"]
            ))
        connection.commit()
        print(f"[{data['timestamp']}] Telemetry logged: Temp={data['water_temp_f']}F, DO={
data['dissolved_oxygen_mg_l']}mg/L, Level={data['water_level_m']}m")
    except Exception as e:
```

```
        print(f"Database sync failed: {e}")
    finally:
        connection.close()

# Example Trigger from incoming Meshtastic Serial payload
if __name__ == "__main__":
    # Test Hex Payload: compTemp=3830 (18.3C/65.0F), compDO=920 (9.2mg/L), compLevel=1150
(1.15m), compTurb=152 (15.2 NTU)
    test_hex = "0EFA0398047E0098"
    decoded_data = decode_meshtastic_telemetry(test_hex)
    if decoded_data:
        sync_to_corteza_crm(decoded_data)
```

VI. Meshtastic CLI Configuration Commands

Deploying a Meshtastic environmental node requires configuring the local device settings via the Meshtastic command line interface (CLI). Hunter's Clemson operations interns can configure the nodes in the lab prior to field installation by connecting the device via USB and running the following commands:

```
# 1. Enable Environment Telemetry Module
meshtastic --set telemetry.environment_measurement_enabled true

# 2. Set Update and Broadcast Interval to 15 Minutes (900 seconds)
meshtastic --set telemetry.environment_update_interval 900

# 3. Configure the Device Serial Module to Receive Custom Hex Packets (RX_HEX)
meshtastic --set serial.enabled true
meshtastic --set serial.baud 9600
meshtastic --set serial.mode TEXTMSG

# 4. Lock the Node in Low-Power Sleep Settings to Protect battery Life
meshtastic --set power.is_power_saving true
meshtastic --set power.on_battery_shutdown_after 0
```

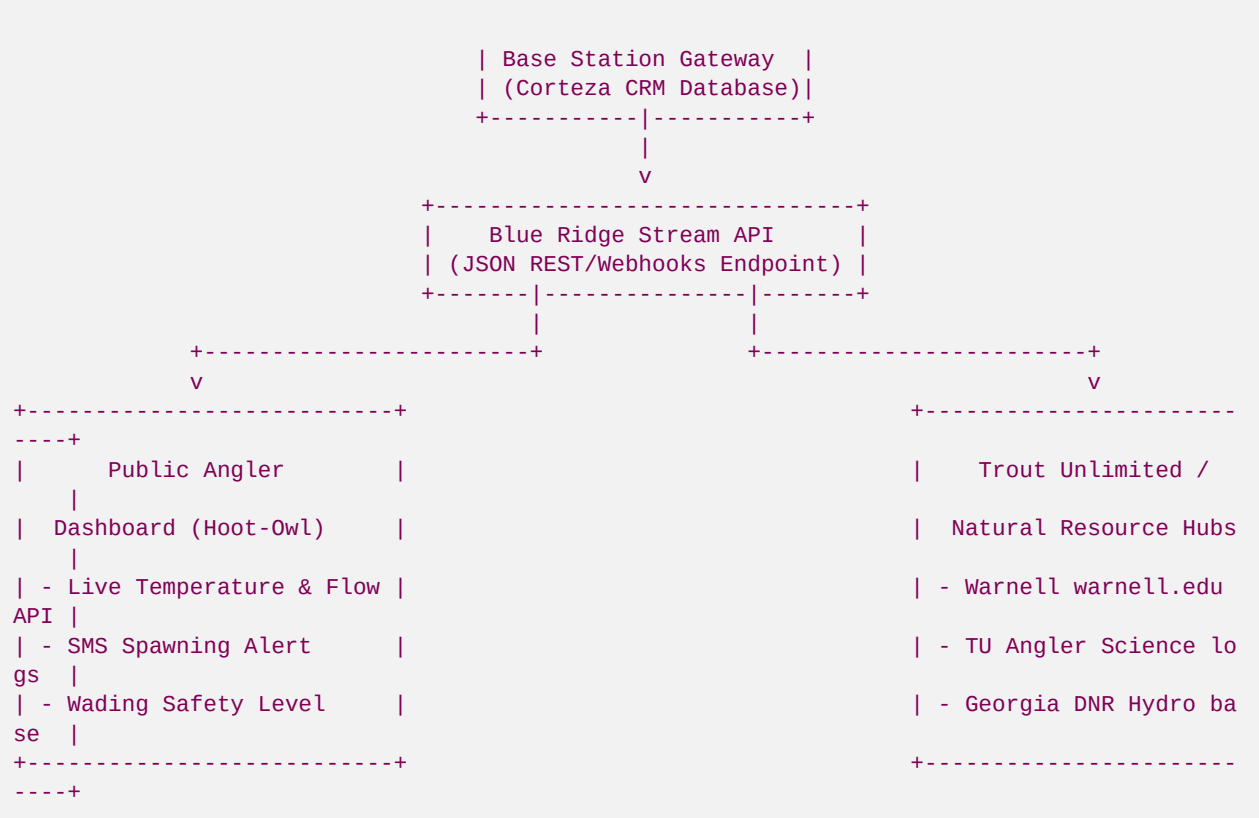
Developed by Blue Ridge Stream Restoration Technical Sourcing Operations. Pushed to remote origin under main.

VII. Angler Science & Community Data-Sharing API

To elevate Hunter Morris’s venture beyond simple compliance and cement his status as Georgia's preeminent angler-builder, we leverage this off-grid LoRa telemetry network to fuel a public-facing **Angler Science & Community Stream Dashboard**.
Trout are coldwater obligate species. When water temperatures exceed 68°F (20°C), trout experience high metabolic stress, and catching them under these conditions often results in high mortality rates. By publishing real-time stream temperature and flow (discharge/level) data, Blue Ridge Stream Restoration provides an invaluable public utility for local fishermen, Trout Unlimited chapters, and state natural resource biologists.

```
                [COMMUNITY DATA-SHARING DISSEMINATION PIPELINE]

                                +-----------------------+
```



1. Public JSON REST API and Wading Safety Gauges

The base station gateway pushes the decompressed stream data to a public-facing REST API hosted on your free GitHub Pages portal. Local outfitter shops, fly guides, and other fishermen can query this endpoint to inspect current wading conditions:

```
{
  "river_section": "Upper Toccoa River - Spawning Reach 2",
  "huc_8": "06020003",
  "status": {
    "hoot_owl_warning": false,
    "wading_safety": "SAFE_TO_WADE"
  },
  "metrics": {
    "water_temperature_f": 58.4,
    "water_temperature_c": 14.67,
    "discharge_stage_m": 0.42,
    "flow_status": "NORMAL_STABLE_FLOW"
  },
  "last_updated": "2026-05-30 19:03:20"
}
```

2. "Hoot Owl" Automated SMS Alerts for Fly Guides

During hot summer months, if the stream temperature exceeds 68°F for more than 2 consecutive hours:

1. **Trigger**: The database sync daemon detects the high temperature threshold.
2. **Notification**: The gateway automatically calls the **Twilio API** to broadcast a SMS text alert to local Georgia guide registry lists:

`"■■ Blue Ridge Conservation Alert: Spawning Reach 2 has exceeded 68°F. Hoot-Owl restrictions recommended. Protect the wild trout."`

3. **Community Impact**: This proactive stewardship highlights Hunter's commitment to wild trout protection and builds massive organic trust among the fly-fishing elite.

3. Open Data Sync with Trout Unlimited and Warnell Warning Networks

To support regional natural resource research, the Python gateway automatically pushes stream data to Trout Unlimited's *Angler Science Database* and the UGA *Warnell School of Forestry & Natural Resources* coldwater tracking network:

```
import requests

TU_INGEST_URL = "https://science.tu.org/api/v1/stream-tracker"
UGA_EXTENSION_URL = "https://warnell.uga.edu/api/coldwater-regime"

def publish_community_angler_science(data):
    """Pushes water temperature and flow data to public ecological databases."""
    payload = {
        "device_id": "blue_ridge_node_101",
        "watershed_huc": "06020003",
        "water_temp_c": data["water_temp_c"],
        "stream_stage_m": data["water_level_m"],
        "dissolved_oxygen_mg_l": data["dissolved_oxygen_mg_l"],
        "reported_at": data["timestamp"]
    }

    try:
        # Push to Trout Unlimited
        response_tu = requests.post(TU_INGEST_URL, json=payload, timeout=5)
        # Push to UGA Forestry Extension
        response_uga = requests.post(UGA_EXTENSION_URL, json=payload, timeout=5)

        if response_tu.status_code == 200 and response_uga.status_code == 200:
            print("Successfully shared hydrological telemetry with Trout Unlimited & UGA W
arnell.")
    except Exception as e:
        print(f"Community open-data sharing failed: {e}")
```

This data-sharing framework transforms private mitigation bank monitoring into a powerful tool for collaborative conservation. It gives fishermen reliable, actionable wading and thermal data while providing biologists with long-term, high-resolution dataset tracking climate resilience in Georgia's coldwater fisheries.